

5.2. Generación de código a partir de diagrama de clases

5.2.1. Activar el Motor Java

5.2.2. Configurar la Ruta de Generación

5.2.3. Preparar el Proyecto para Java

5.2.4. Generar el Código

5.2.5. Verificar los archivos físicos

5.2.6. Preguntas frecuentes

Modelio es una herramienta profesional muy potente, pero su curva de aprendizaje inicial es muy inclinada porque **no viene pre-configurada para generar código nada más instalarla**. Hay que "preparar" el entorno.

Vamos a hacerlo **desde cero, paso a paso y sin dar nada por supuesto**. Sigue esta guía al pie de la letra.

5.2.1. Activar el Motor Java (Java Designer)

Si este módulo no está cargado y activado ("Enable"), Modelio es solo una herramienta de dibujo.

1. Abre tu proyecto en Modelio.
 2. Ve al menú superior: **Configuration** $\rightarrow$ **Modules...**
 3. Se abrirá una ventana llamada "Project modules".
 - **Escenario A (Ya aparece en la lista):** Si ves **JavaDesigner** en la lista, mira la columna **Enable**. Si no está marcada, haz clic en ella. Y si lo está, ya está activado el Motor Java.
 - **Escenario B (La lista está vacía o no está JavaDesigner):**
 1. Haz clic en el botón **Add...** (parte superior derecha de esa ventana).
 2. Elige **JavaDesigner**. Doble clic.
 3. Ahora aparecerá en la lista. Dale al botón **Deploy in the project** (o Start).
 4. Cierra la ventana de módulos.
-

5.2.2. Configurar la Ruta de Generación (Dónde se guardan los archivos)

Modelio necesita saber explícitamente dónde escribir.

1. En el **Menú principal**, haz clic en la **Configuration** -> **Modules**.
 2. Selecciona **Java Designer**. Se abrirán los parámetros del módulo para su configuración
 3. Configura estos dos campos obligatorios de la categoría **Directories**:
 - **Code Generation Path:** Aquí debes decirle dónde guardar los **.java**.
 - Escribe manualmente: **\$(Project)/src**
 - *Explicación:* Esto creará una carpeta **src** dentro de la carpeta del proyecto.
 - **JDKPath (Opcional):** Para *generar* código fuente no hace falta rellenarlo (Modelio sabe escribir texto Java). Solo es necesario si quisieras *compilar* (.class) desde dentro de Modelio. No usaremos Modelio para eso.
-

5.2.3. Preparar el Proyecto para Java

Ahora tenemos que decirle a Modelio qué partes del diagrama son "código real".

1. En el **Model Browser**, localiza el **Paquete** donde dibujaste tus clases (ej: **ClasesAcademia** o **Package1**, normalmente es el icono amarillo que representa un directorio).
2. Haz clic en el Paquete.
3. En el menu contextual selecciona **Add stereotype**.
4. Escribe o busca: **JavaPackage**. Selecciónalo y añádelo.
 - *Efecto visual:* El icono del paquete cambiará ligeramente (quizás aparezca una imagen de la taza de Java). Esto confirma que Modelio lo tratará como una estructura de código.
5. Haz clic en una clase.
6. En el menu contextual selecciona **Add stereotype**.
7. Escribe o busca: **JavaClass**. Selecciónalo y añádelo.
 - *Efecto visual:* El icono del paquete cambiará ligeramente (quizás aparezca una imagen de la taza de Java). Esto confirma que Modelio lo tratará como una estructura de código.
8. Repite este paso para cada clase de la que queramos generar código.

No es estrictamente necesario añadir estereotipos manuales a los atributos y relaciones, ya que Modelio es bastante listo infiriendo el código.

Sin embargo, para que el código sea "**profesional**" (es decir, que use **private**, que tenga **Getters/Setters** y que las listas sean **ArrayList**), hay que tocar ciertas configuraciones que actúan como "estereotipos técnicos".

A continuación se detallan algunos aspectos a configurar para que el código salga limpio.

9. En los Atributos (Variables)

No necesitas ponerles estereotipos (como `<<field>>`), pero sí **dos cosas fundamentales**:

- La Visibilidad (Private) Asegúrate de que en el diagrama los atributos tengan el signo menos (**private**).
 - **Efecto en código:** Modelio generará `private String nombre;`.
 - Si lo dejan público (+), generará `public String nombre;`, lo cual rompe el principio de encapsulamiento.
- Los Getters y Setters (Acceso) En lugar de escribir métodos a mano o buscar un estereotipo, Modelio tiene una función automatizada.
 - Selecciona el atributo (ej: **nombre**).
 - En el panel **Properties**, pestaña **Java**, hay una opción llamada **"Getter/Setter"** o **"Accessors"**.

- Márcala.
- **Resultado:** Modelio generará automáticamente `getNombre()` y `setNombre()` en el archivo `.java`.

10. En las Relaciones (Asociaciones)

Aquí es donde está la clave para que genere `List<Estudiante>` correctamente. No se usan estereotipos visuales sobre la línea, se configuran las **Propiedades del Extremo de la Asociación**.

Cuando tienes una relación 1 a muchos (`0..*`), Modelio por defecto usa `java.util.List` y `java.util.ArrayList`. Pero si quieres afinar, haz esto:

- Selecciona la línea de asociación.
- Ve a **Properties** y busca la pestaña **Java** (o Java Designer).
- Verás dos columnas (una para cada extremo de la relación, *Association End*). Fíjate en el extremo que tiene el `*` (el lado "Muchos"). Aquí puedes configurar dos cosas vitales sin usar estereotipos:
 - **Type of collection interface:** Por defecto es `List`. Puedes cambiarlo a `Set` (si quieres enseñar que no hay duplicados) o `Collection`.
 - **Implementation class:** Por defecto es `ArrayList`. Puedes cambiarlo a `LinkedList` o `HashSet`. **Recomendación:** Déjalo por defecto (`List` / `ArrayList`), es lo estándar y funciona bien.

11. Los Estereotipos que Sí son útiles (y necesarios)

Aunque para atributos y relaciones no hacen falta, hay 3 estereotipos de clase que **debemos conocer** para modelar correctamente en Java:

A. `<<interface>>` (Para Interfaces)

Si quieres definir un contrato (ej: una interfaz `Imprimible` con un método `imprimir()`).

- **Cómo:** Existe un elemento diferente para los Interfaces en el diagrama, un círculo, luego deberemos añadir el estereotipo `<<interface>>`.
- **Código:** Generará `public interface Imprimible { ... }` en lugar de `class`.

B. `<<enumeration>>` (Para Enums)

Si tienes una clase `DiasSemana` o `TipoDepartamento`.

- **Cómo:** Existe un elemento diferente para los Enumerations en el diagrama, un cuadradito con lista de números, luego deberemos añadir el estereotipo `<<enumeration>>`.
- **Código:** Generará `public enum DiasSemana { ... }`.

C. `<<JavaNoCode>>` (Para ignorar elementos)

A veces dibujas una nota o una clase auxiliar en el diagrama para explicar algo, pero no quieres que se convierta en un archivo `.java`.

- **Cómo:** No añades este estereotipo desde la lista de Java Designer.
- **Código:** Modelio ignorará ese elemento al generar.

5.2.4. Generar el Código

1. Haz **clic derecho** sobre tu paquete (**ClasesAcademia**).
 2. En el menú contextual, ve a **Java Designer** $\rightarrow$ **Generate**.
 - *Lo que debe pasar:* Verás una barra de progreso breve. Si la configuración de la Fase 2 está bien, no dará error.
 3. Visualiza el Código:
 - Haz clic en una clase.
 - En el menu contextual selecciona **Java Designer** $\rightarrow$ **Edit**.
 - Aparecerá una nueva pestaña en tu interfaz
-

5.2.5. Verificar los archivos físicos

Los archivos ya deben existir en tu disco duro si la Fase 2 funcionó.

1. Abre el Explorador de Archivos de Windows.
 2. Ve a **C:\Users\TuUsuario\modelio\workspace\NombreDeTuProyecto**.
 3. Entra en la carpeta **src**.
 4. Dentro estarán **Profesor.java**, **Curso.java** y **Estudiante.java**.
 5. Repite la generación de código refernciando (en los parámetros de configuración del Modulo **JavaDesigner**) en Directories, el prámetro Code Generation Path, indicando la ruta del proyecto generado en VSC (Java: Create Java Project), por ejemplo:
C:\Users\nombreusuario\Documents\WorkSpace\AppAcademia\src.
-

5.2.6. Preguntas frecuentes

Pregunta1: Sustitución del JDK ¿?

El código que genera Modelio son archivos de texto plano (**.java**). No es un binario compilado ni un código propietario extraño. Por tanto:

1. **Es código Java estándar:** Puedes coger esos archivos **.java** generados, llvátelos a otro ordenador que tenga instalado el JDK 17, el 21 o el 8, y funcionarán perfectamente (siempre que no uses características muy nuevas en una versión vieja).
2. **Configuración en Modelio:** Lo único que debes vigilar es la **compatibilidad de sintaxis**.
 - Si en Modelio (Propiedades del módulo Java Designer) tienes configurada la **Java Compatibility** en **Java 8**, el código generado será compatible con cualquier JDK desde el 8 en adelante.

Atencion: Lo normal es generar el código en Modelio y luego abrir esa carpeta **src** con un IDE real como **VSC, IntelliJ IDEA, Eclipse o NetBeans**. Esos IDEs usarán su propio JDK configurado, ignorando el de Modelio.

Pregunta2: ¿Para qué sirven las anotaciones @objid?

Esta es la duda número 1 al ver el código sucio. Verás algo así:

```
@objid ("34f5d2e1-88bc-...")
public class Profesor { ... }
```

Explicación sencilla: El `@objid` es como el **DNI (Documento de Identidad)** o la matrícula interna de ese elemento dentro de la base de datos de Modelio.

¿Para qué sirve exactamente? Sirve para mantener la **Sincronización (Round-Trip Engineering)**.

1. **Del Diagrama al Código:** Modelio sabe que la clase "Profesor" del dibujo corresponde al archivo `Profesor.java` gracias a ese ID.
2. **Del Código al Diagrama (Lo importante):** Si modificamos el código Java directamente (por ejemplo, cambiáis el nombre de un atributo en VSC), al volver a Modelio y darle a "Actualizar", Modelio busca ese `@objid`. Si lo encuentra, actualiza el dibujo. Si no lo encuentra, se pierde y cree que es una clase nueva.

¿Son obligatorias?

- **Para Java: NO.** El compilador de Java las ignora. El programa funcionará igual si las borras.
- **Para Modelio: SÍ.** Si las borras, rompes el vínculo entre el dibujo y el código. Ya no podrás actualizar el modelo automáticamente desde el código.

¿Se pueden quitar para que el código quede "limpio"? Sí, se puede configurar Modelio para que no las genere, o para que las ponga como comentarios Javadoc en lugar de anotaciones (que ensucian menos).

Cómo hacerlo (Opcional):

1. Ve a **Configuration > Modules > Java Designer**.
2. Busca la opción **GenerationMode**.
3. Cámbiala de `RoundTrip` a `Release` o `ModelDriven`.
 - Esto generará el código limpio, sin `@objid`, pero **perderás la capacidad de sincronizar cambios futuros** desde el código hacia el modelo.

Recomendación: De momento ignorad las anotaciones, que son "etiquetas de rastreo" de la herramienta. No cambiéis el **GenerationMode**.